TRIP PLANNER PWA

Répartition des modules et logique métier

Un module par membre : ce qu'il gère et les règles métier qu'il applique.

Membre	Module	Collection	Fonctionnalité avancée
Membre 1	Gestion des utilisateurs	`users`	Export PDF de l'itinéraire et liens de partage publics
Membre 2	Gestion des voyages	`trips`	Synchronisation offline-first (service worker + IndexedDB)
Membre 3	Gestion des lieux et de l'itinéraire	`places`	Carte interactive avec route quotidienne
Membre 4	Gestion des dépenses	`expenses`	Conversion multi-devises et graphiques de budget
Membre 5	Gestion de la checklist	`checklist_items (+ checklist_templates)`	Checklist générée à partir des préférences du voyage
Membre 6	Gestion du journal de voyage	`journal_entries`	Générateur d'itinéraire par IA (à réaliser)
Membre 7	Gestion du forum communautaire	`forum_posts (+ forum_replies)`	Recherche plein texte et notifications Web Push

Règles communes : chaque membre est le seul à écrire dans sa collection ; les autres modules passent par son API. Toutes les routes sont sous `/api/v1` et les permissions sont vérifiées à chaque appel via la liste des membres du voyage.

MEMBRE 1

Module 1 — Gestion des utilisateurs

Collection : **users** · Fonctionnalité avancée : **Export PDF de l'itinéraire et liens de partage publics** · Rôle transverse : **Authentification et mise en page commune (AppShell)**

Gérer le cycle de vie complet d'un compte : inscription, connexion, session sécurisée, mot de passe oublié, profil, suppression du compte et administration (rôles, désactivation, statistiques). Le module fournit aussi le middleware d'authentification utilisé par tous les autres modules, ainsi que deux fonctions de diffusion : l'export PDF d'un voyage et le lien public en lecture seule.

Inscription et connexion

- Le corps de chaque requête est validé avec **zod** : nom de 2 à 80 caractères, email valide, mot de passe d'au moins 8 caractères.
- Un email déjà utilisé renvoie **409** avec une erreur sur le champ `email`.
- Le mot de passe est haché avec **bcrypt (coût 12)** ; le hash n'est jamais renvoyé (filtré par `toJSON`).
- En cas d'échec de connexion, le message est volontairement générique (« Invalid email or password ») pour ne pas révéler si le compte existe.
- Un compte désactivé ne peut pas se connecter (**403**).
- Les routes d'authentification sont limitées à **20 tentatives / 15 min / IP** (réponse 429).

Captcha (protection contre les robots)

- Captcha **auto-hébergé** : l'API génère l'image (*svg-captcha*) ; aucun service externe ni clé n'est nécessaire, et le navigateur ne contacte que notre API (règle du cahier des charges).
- `GET /auth/captcha` renvoie un identifiant et une image SVG de **5 caractères** dessinés en tracés (pas en texte lisible dans le code), sans caractères ambigus (0/O, 1/l/I). Limité à 30 images par minute et par IP.
- Le serveur ne garde que le **hash SHA-256** de la réponse, **5 minutes** au maximum. La comparaison ignore la casse et se fait en temps constant.
- Chaque captcha est **à usage unique** : il est supprimé après une tentative, bonne ou mauvaise. Le formulaire charge alors une nouvelle image.
- Obligatoire à l'**inscription** et pour le **mot de passe oublié**. Réponse fausse ou absente → 400, erreur sur le champ `captcha` et `captchaRequired: true`.
- **Connexion** : après **3 échecs en 15 minutes** pour le même email, la connexion exige un captcha, même avec le bon mot de passe. Le compteur est lié à l'**email** (pas à l'IP), donc changer ou falsifier son IP ne permet pas de l'éviter ; une connexion réussie le remet à zéro.
- Désactivable avec `CAPTCHA_ENABLED=false` (tests automatisés). Limite : les captchas en attente sont gardés en mémoire, donc avec une seule instance de l'API.

Adresse IP et historique des connexions

- À chaque **connexion** ou **inscription** réussie, l'API enregistre l'**adresse IP**, l'appareil (navigateur et système, déduits du User-Agent) et la date.
- Les **10 dernières** connexions sont conservées (`loginHistory`, la plus récente en premier) ; `lastLoginIp` garde la dernière.
- Le rafraîchissement du jeton n'est pas une nouvelle connexion : il n'est pas enregistré.
- L'utilisateur voit ses connexions dans son profil (« Recent connections ») et est invité à changer son mot de passe s'il ne reconnaît pas l'une d'elles. L'administrateur voit la dernière IP de chaque compte.
- L'IP est lue dans l'en-tête `X-Forwarded-For` selon `TRUST_PROXY`. Le proxy Next.js n'ajoute pas d'entrée : en production, un reverse proxy (nginx, hébergeur) placé devant Next fournit l'IP réelle ; en local elle vaut 127.0.0.1.

Une valeur qui n'est pas une IP valide est ignorée.
- Donnée **informative** : aucune décision de sécurité ne repose sur l'IP, puisqu'un en-tête peut être falsifié sans reverse proxy.

Session : access token + refresh token avec rotation

- **Access token** JWT valable **15 minutes**, contenant l'id et le rôle, envoyé dans l'en-tête `Authorization: Bearer`. Côté client il reste en mémoire (pas dans localStorage) pour limiter le risque XSS.
- **Refresh token** JWT valable **7 jours**, stocké dans un cookie **httpOnly** limité au chemin `/api/v1/auth`. Son identifiant unique (`jti`) est enregistré sur l'utilisateur.
- **Rotation à chaque usage** : `POST /auth/refresh` marque l'ancien jti comme « tourné » et en émet un nouveau.
- **Détection de vol** : si un jeton déjà tourné est présenté plus de 15 s après sa rotation, toutes les sessions de l'utilisateur sont révoquées.
- Une **fenêtre de grâce de 15 s** évite de déconnecter l'utilisateur quand deux onglets (ou le service worker) rafraîchissent en même temps.
- Le middleware `requireAuth` vérifie le JWT puis relit `role` et `isActive` en base : une désactivation ou un changement de rôle prend effet immédiatement.
- Le client fait un rafraîchissement « single-flight » (une seule requête partagée) et rejoue automatiquement une requête refusée en 401.

Mot de passe oublié

- La réponse est identique que le compte existe ou non (protection contre l'énumération d'emails).
- Le jeton est une valeur aléatoire de **32 octets** ; seul son **hash SHA-256** est stocké, avec une expiration d'**1 heure**.
- Le lien est envoyé par SMTP si configuré, sinon affiché dans la console de l'API (développement).
- La réinitialisation invalide toutes les sessions ouvertes (refreshTokens vidés).
- Le changement de mot de passe depuis le profil exige l'ancien mot de passe.

Suppression du compte (règle 2 du modèle de données)

- Le mot de passe est redemandé.
- Dans **une transaction** : les voyages dont il est propriétaire sont supprimés en cascade, il est retiré des voyages des autres, ses posts et réponses du forum sont **anonymisés** (auteur = null, affiché « Deleted user »), il est retiré des abonnés des fils, puis le compte est supprimé.
- Les fils de discussion restent donc lisibles pour la communauté.

Administration

- Liste paginée des comptes avec recherche par nom ou email.
- Désactiver / réactiver un compte : la désactivation ferme aussi toutes ses sessions.
- Passer un membre administrateur (ou l'inverse). Un administrateur ne peut pas modifier son propre compte depuis cet écran.
- Statistiques : nombre d'utilisateurs, actifs et nouveaux sur 7 jours, voyages, lieux, dépenses, articles de checklist, entrées de journal, posts, réponses, signalements ouverts, top 5 des pays de destination.

Logique de la fonctionnalité avancée — Export PDF de l'itinéraire et liens de partage publics

Export PDF — `GET /trips/:id/export` (membre du voyage) : le PDF est généré **côté serveur** avec *pdfkit* et envoyé en flux. Il contient l'en-tête (titre, destination, dates, nombre de jours, budget, participants), puis chaque jour avec ses lieux triés par ordre (heure, nom, adresse, catégorie, durée, coût estimé, notes), une section « non planifiés » et la date de génération.

Lien de partage — `POST /trips/:id/share` (propriétaire uniquement) crée un `shareToken` aléatoire de **32 octets** (64 caractères hexadécimaux), avec un index unique « sparse ». L'appel est idempotent : s'il existe déjà, le même lien est renvoyé.

`DELETE /trips/:id/share` **révoque** le lien : l'ancienne URL renvoie 404.

`GET /share/:token` est **public** : le format du jeton est vérifié, puis seules les informations non sensibles sont renvoyées (titre, destination, dates, couverture, prénoms des participants, lieux). Les dépenses et le journal restent privés.

MEMBRE 2

Module 2 — Gestion des voyages

Collection : **trips** · Fonctionnalité avancée : **Synchronisation offline-first (service worker + IndexedDB)** · Rôle transverse : **Déploiement et pipeline CI**

Gérer le voyage, objet central de l'application : informations générales, budget, devise de référence, préférences qui pilotent la checklist, participants et droits. Le module garantit aussi que l'application continue de fonctionner sans réseau : lecture complète des voyages ouverts et écritures mises en file d'attente puis rejouées à la reconnexion.

Création

- Validation zod : titre, destination, pays, dates (fin ≥ début), budget ≥ 0, devise, préférences avec valeurs autorisées uniquement (enums).
- Le créateur devient **propriétaire** et premier membre (`role: owner`).
- Dans **la même transaction**, la checklist est générée à partir des préférences (appel au module 5).
- **Idempotence** : le client peut fournir l'`_id` (créé hors ligne). Si ce voyage existe déjà et que l'utilisateur en est membre, il est simplement renvoyé, ce qui évite les doublons quand la file est rejouée deux fois.

Lecture et permissions

- Un utilisateur ne voit que les voyages où il figure dans `members` ; la vérification se fait à **chaque appel** (`loadTripForMember`).
- Les modules enfants stockent la référence du voyage : une permission = une seule lecture du voyage (règle 3 du modèle).
- Tout membre peut lire et modifier le voyage et son contenu.
- Seul le **propriétaire** peut supprimer le voyage, inviter, retirer un participant et gérer le lien de partage.
- Un participant peut quitter le voyage ; le propriétaire ne peut pas (il doit le supprimer).
- Liste paginée avec recherche (titre, destination) et tri (date de début, création, titre).

Participants

- Invitation par email : le compte doit exister et être actif, sinon 404 avec un message explicite.
- Pas de doublon : inviter quelqu'un déjà présent renvoie 409.

Modification et suppression

- Chaque modification accepte `clientUpdatedAt` (heure réelle de la modification par l'utilisateur). Si le serveur a une version plus récente, il refuse avec **409** et renvoie sa version : **last-write-wins**.
- Si les préférences, les dates ou le pays changent, la checklist est **resynchronisée** automatiquement.
- Suppression en **cascade dans une transaction** : lieux, dépenses, checklist, journal, puis le voyage (opérations séquentielles, MongoDB n'autorisant pas les opérations parallèles dans une même session).
- `GET /trips/:id/full` renvoie le voyage et tout son contenu en une requête, pour en garder une copie hors ligne.

Logique de la fonctionnalité avancée — Synchronisation offline-first (service worker + IndexedDB)

Copie locale (IndexedDB, Dexie) : chaque voyage ouvert est téléchargé avec ses lieux, dépenses, checklist et journal. Les écrans lisent cette copie (requêtes « live »), donc ils fonctionnent sans réseau.

Écriture optimiste + file d'attente (outbox) : chaque création, modification ou suppression est appliquée immédiatement en local puis enregistrée dans la table `outbox` : entité, opération, voyage, document, contenu, heure de la modification.

Identifiants générés côté client (ObjectId) : un élément créé hors ligne a déjà son identifiant définitif, ce qui rend le rejeu idempotent et permet de chaîner « créer puis modifier ».

Rejeu dans l'ordre d'enregistrement :

– succès → l'op est retirée et la version serveur remplace la copie locale (sauf s'il reste d'autres ops sur ce document) ;

– erreur réseau, 5xx ou 429 → arrêt et nouvel essai plus tard ;

– 401 → l'indicateur affiche « Log in to sync » ;

– **409** → la version du serveur est gardée et un message prévient l'utilisateur ;

– 404 → l'élément est supprimé localement ;

– autre erreur 4xx → l'op est abandonnée.

Déclencheurs du rejeu : retour du réseau, retour sur l'onglet, test de connectivité toutes les 15 s, bouton de l'indicateur, et **Background Sync** : le service worker rejoue la même file même quand l'application est fermée (Chrome/Edge).

Un **verrou** (Web Locks API) empêche la page et le service worker de rejouer la file en même temps.

Fusion au téléchargement : un document qui a encore des modifications en attente n'est jamais écrasé par la version serveur ; les documents supprimés sur le serveur sont retirés localement.

Service worker (`public/sw.js`) : stratégies de cache (voir tableau ci-dessous), page de secours `/offline`, nettoyage des anciens caches.

Indicateur de synchro visible en permanence : Synced, Offline · N pending, Syncing…, Log in to sync.

Installation : manifest (icônes 192/512 et maskable, mode standalone), bouton « Install » personnalisé ; sur iOS, guide « Ajouter à l'écran d'accueil ».

MEMBRE 3

Module 3 — Gestion des lieux et de l'itinéraire

Collection : **places** · Fonctionnalité avancée : **Carte interactive avec route quotidienne**

Construire l'itinéraire jour par jour : ajouter des lieux (recherche d'adresse ou clic sur la carte), les affecter à un jour, fixer heure, durée et coût estimé, les réordonner par glisser-déposer, et visualiser sur une carte le parcours de chaque journée avec les temps de trajet entre les étapes.

Création et modification

- Seuls les membres du voyage peuvent lire ou écrire (vérification via le voyage).
- À la création, le lieu est placé **à la fin de son jour** (ordre = dernier ordre du jour + 1).
- Si on change le jour sans préciser l'ordre, le lieu est ajouté **à la fin du nouveau jour**.
- Création idempotente avec un id client ; modification protégée par last-write-wins (409 si plus ancienne que le serveur).
- La suppression est idempotente : supprimer deux fois n'est pas une erreur, ce qui est utile pour le rejeu hors ligne.

Organisation de la journée

- `PUT /places/reorder` reçoit le jour et la nouvelle liste ordonnée ; il vérifie que tous les lieux appartiennent au voyage puis met à jour les ordres en une opération groupée.
- Dans l'interface, le glisser-déposer envoie une modification d'ordre par lieu déplacé : ce mécanisme fonctionne aussi hors ligne.
- Accessibilité : réordonnancement au clavier, et liste « Move to day » comme alternative au glisser-déposer.
- Liste paginée à **50** lieux, avec filtres par jour (ou non planifiés) et par catégorie, et recherche par nom.
- La durée estimée d'une journée = somme des durées de visite + trajets (le plus court entre marche et voiture).

Logique de la fonctionnalité avancée — Carte interactive avec route quotidienne

Carte Leaflet + tuiles OpenStreetMap. Marqueurs **numérotés et colorés par jour** (10 couleurs) ; les lieux non planifiés sont en gris.

Filtre par jour : on n'affiche que les marqueurs et la route du jour choisi. La carte se recadre automatiquement sur les lieux visibles.

Route du jour — `GET /trips/:id/route?day=N` : l'API appelle **OSRM** avec les étapes dans l'ordre et renvoie le tracé de la route, la distance et la durée en voiture de chaque tronçon.

Temps à pied estimé : distance à vol d'oiseau (formule de Haversine) × 1,25 à 4,5 km/h.

Mode dégradé : si OSRM ne répond pas (4 s), l'API calcule une estimation (distance × 1,3 à 40 km/h). Hors ligne, le navigateur fait le même calcul et affiche « estimate ».

Les routes sont mises en **cache 6 h** côté serveur ; les tuiles déjà vues restent disponibles hors ligne grâce au service worker.

Géocodage — `GET /geocode?q=` : recherche d'adresse via Nominatim, appelée par l'API (le client ne contacte jamais un service externe directement), avec cache et en-tête User-Agent. Sans réseau, on place le lieu en cliquant sur la carte.

MEMBRE 4

Module 4 — Gestion des dépenses

Collection : **expenses** · Fonctionnalité avancée : **Conversion multi-devises et graphiques de budget**

Suivre les dépenses partagées du groupe : saisie dans n'importe quelle devise, conversion automatique dans la devise du voyage, répartition entre participants, photo du reçu, suivi du budget avec alerte à 80 %, graphiques, et calcul de « qui doit combien à qui ».

Saisie

- Par défaut, le **payeur** est l'utilisateur connecté et la dépense est **partagée entre tous les membres**.
- Le payeur et chaque participant doivent être **membres du voyage** (sinon 400).
- La dépense est **stockée deux fois** : montant d'origine et montant converti, avec le taux utilisé. Le budget reste donc juste même si les taux changent ensuite.
- La conversion est recalculée seulement si le montant ou la devise change.
- Arrondis : montants à 2 décimales, taux à 6 décimales.
- Création idempotente (id client), modification en last-write-wins, suppression idempotente.

Taux de change

- Les taux sont récupérés depuis une API publique (open.er-api.com) avec la devise du voyage comme base, puis **mis en cache 24 h côté serveur**.
- Si l'API est indisponible : **table de secours** de 34 devises (base EUR), et nouvel essai au bout d'une heure.
- Devise inconnue → 400 avec une erreur sur le champ `currency`.
- `GET /rates` est public : le client garde les derniers taux pour faire des estimations hors ligne.

Budget, soldes et remboursements

- Total dépensé = somme des `amountInBase` ; pourcentage = total / budget.
- **Alerte à 80 %** du budget (orange), et « Budget exceeded » au-delà de 100 % (rouge).
- Répartition par catégorie et par jour pour les graphiques.
- **Soldes** : le payeur est crédité du montant, chaque participant est débité de sa part égale (montant / nombre de participants).
- **Remboursements** : algorithme glouton — le plus gros débiteur rembourse le plus gros créancier, et ainsi de suite, ce qui limite le nombre de virements.

Hors ligne

- Le résumé est recalculé dans le navigateur avec le même algorithme : il fonctionne sans réseau.
- Une dépense en devise étrangère saisie hors ligne est convertie avec les **derniers taux en cache** et affiche le badge « estimate » ; le serveur recalcule la vraie valeur à la synchronisation.
- La photo du reçu nécessite une connexion (envoi d'image).

Logique de la fonctionnalité avancée — Conversion multi-devises et graphiques de budget

Conversion automatique à la saisie, avec double stockage (devise d'origine et devise du voyage).

Taux mis en cache une fois par jour côté serveur, avec repli en cas de panne.

Graphiques (Recharts) : **camembert par catégorie** et **histogramme par jour**, avec une description textuelle pour les lecteurs d'écran.

Barre de progression du budget avec les seuils 80 % et 100 %.

Tableau « To settle up » : qui doit combien à qui.

MEMBRE 5

Module 5 — Gestion de la checklist

Collection : **checklist_items (+ checklist_templates)** · Fonctionnalité avancée : **Checklist générée à partir des préférences du voyage**

Produire automatiquement une liste de bagages et de préparatifs adaptée au voyage (type, activités, hébergement, transport, voyageurs, besoins spécifiques, saison, durée, destination), puis laisser l'utilisateur la compléter, la modifier et cocher ce qui est prêt. Les règles sont des données (collection de modèles), pas du code figé.

Contexte calculé pour chaque voyage

- Les **préférences** saisies à la création : tripType, activities, accommodation, transport, travellers, specialNeeds.
- **season** (dérivé) : mois de départ + hémisphère du pays de destination. Hiver = décembre à février au nord ; les saisons sont inversées pour les pays de l'hémisphère sud.
- **duration** (dérivé) : nombre de jours calendaires, départ et retour inclus.
- **isAbroad** : pays de destination ≠ pays de résidence du propriétaire (si le pays de résidence est inconnu, on considère le voyage à l'étranger).

Moteur de règles (data-driven)

- Chaque modèle contient une condition et une liste d'articles ; il « se déclenche » si la condition est vraie.
- Opérateurs : `always`, `eq`, `neq`, `in`, `includes` (pour les listes), `gt`, `gte`, `lt`, ainsi que les combinaisons `any` (OU) et `all` (ET). Les champs imbriqués sont permis, par exemple `travellers.infants`.
- Exemple : `{ any: [ {field:"tripType", op:"eq", value:"mountain"}, {field:"activities", op:"includes", value:"hiking"} ] }`.
- Les articles sont **dédupliqués par libellé** (sans tenir compte de la casse) : si deux règles ajoutent « Sunscreen », il n'apparaît qu'une fois.
- **Règle de quantité** : un article comptable (sous-vêtements, chaussettes, t-shirts) vaut **min(durée + 1, 10) × nombre d'adultes** ; les autres valent 1.
- 18 modèles sont chargés par `npm run seed` : les 11 du cahier des charges et 7 complémentaires (tenue formelle, conduite à l'étranger, tenue modeste ou pèlerinage, business, ski, natation, été). L'application ne les modifie jamais.

Régénération (la règle importante)

- Elle se déclenche à la **création** du voyage, à chaque **changement de préférences, de dates ou de pays**, et avec le bouton « Regenerate ».
- Elle **ajoute** les articles manquants ; un article ajouté à la main avec le même libellé compte comme présent.
- Elle **met à jour** la quantité et la catégorie des articles générés que l'utilisateur n'a pas touchés.
- Elle **supprime** les articles générés qui ne correspondent plus, **sauf s'ils ont été modifiés ou cochés**.
- Elle ne touche **jamais** aux articles ajoutés à la main.
- Modifier le libellé, la quantité ou la catégorie d'un article généré le marque `isEdited` : il est protégé.
- La génération se fait **côté serveur** : le même voyage donne la même liste sur tous les appareils.

Logique de la fonctionnalité avancée — Checklist générée à partir des préférences du voyage

Exemple : Crète, 7 jours en juillet, type beach, activité diving, 1 adulte, pays de résidence Tunisie. Résultat : 25 articles.

– règles *Always*, *beach*, *diving*, *plane*, *abroad* et *summer* déclenchées ;

– chaussettes, t-shirts et sous-vêtements ×8 (7 + 1).

Les écrans affichent pour quel contexte la liste a été générée : saison, durée, type, « à l'étranger », activités.

Hors ligne : on peut cocher, ajouter et modifier (file d'attente). La régénération demande une connexion, puisqu'elle est faite par le serveur.

MEMBRE 6

Module 6 — Gestion du journal de voyage

Collection : **journal_entries** · Fonctionnalité avancée : **Générateur d'itinéraire par IA (à réaliser)**

Permettre aux voyageurs de garder une trace de chaque journée : entrées datées avec texte, photos, humeur, lieu de l'itinéraire et position GPS, lisibles et modifiables même sans réseau. Le module porte aussi l'envoi d'images (utilisé pour les reçus) et la future génération d'itinéraire par IA.

Règles du journal

- Tous les membres du voyage lisent toutes les entrées et peuvent en écrire.
- Seul l'**auteur** peut modifier ou supprimer son entrée (403 sinon) ; l'interface masque les boutons pour les autres.
- Le lieu rattaché doit appartenir au **même voyage** (sinon 400).
- Tri par date décroissante ; filtre possible par lieu.
- Création idempotente (id client), modification en last-write-wins, suppression idempotente.

Hors ligne (exigence du cahier des charges)

- Les entrées sont **lisibles et modifiables sans réseau** : texte, humeur, lieu et position GPS (API de géolocalisation du navigateur).
- Elles sont mises en file d'attente et envoyées à la reconnexion.
- L'ajout de **photos** nécessite une connexion (l'interface l'indique).

Envoi d'images (module uploads)

- `POST /uploads?folder=journal|receipts` : jusqu'à 6 images par envoi, 5 Mo maximum chacune, formats JPEG, PNG, WebP, GIF, AVIF.
- Stockage sur **Cloudinary** (converties en WebP) si les clés sont configurées, sinon dans le dossier `uploads/` de l'API.
- Affichage en tailles adaptées (paramètres Cloudinary de largeur et qualité automatique).

Logique de la fonctionnalité avancée — Générateur d'itinéraire par IA (à réaliser)

À réaliser — générateur d'itinéraire par IA (spécification à suivre) :

– un endpoint, par exemple `POST /trips/:tripId/ai-itinerary`, reçoit la destination, le nombre de jours et les centres d'intérêt ;

– l'API appelle le modèle d'IA **côté serveur** (clé dans les variables d'environnement, jamais dans le client).

La réponse doit être demandée en **JSON structuré** (jours → lieux : nom, latitude, longitude, catégorie, heure, durée, coût) puis **validée avec zod**. Les lieux invalides sont rejetés.

Après validation par l'utilisateur (aperçu), les lieux sont insérés comme de **vrais lieux** du module 3, modifiables ensuite, en passant par la logique du module 3 (règle « on n'écrit pas dans la table d'un autre membre »).

Une **limite d'appels** (rate limiting) s'applique à cet endpoint, comme exigé par la sécurité du cahier des charges.

Risque identifié : l'IA lente ou indisponible pendant la soutenance. Mitigation prévue : itinéraires déjà générés pour **deux destinations de démo**, enregistrés en base.

MEMBRE 7

Module 7 — Gestion du forum communautaire

Collection : **forum_posts (+ forum_replies)** · Fonctionnalité avancée : **Recherche plein texte et notifications Web Push** · Rôle transverse : **Design system et composants partagés**

Offrir un espace public où les voyageurs posent des questions sur des destinations, partagent des conseils et des récits, se répondent, votent et signalent les abus, avec une modération par les administrateurs. C'est la seule partie publique de l'application : elle lui donne de la valeur même en dehors d'un voyage en cours.

Lecture et recherche

- Lecture **publique** : les visiteurs consultent sans compte.
- Un post masqué disparaît des listes et n'est visible que par son auteur et les administrateurs.
- **Recherche plein texte** sur le titre (poids 3) et le corps (poids 1) grâce à un **index texte MongoDB** ; filtres par catégorie et par tag de destination.
- Tris : pertinence, récents, les plus votés, récemment actifs. Liste paginée.
- Les tags populaires sont calculés par agrégation.

Publication et discussion

- Écrire demande un compte et une **connexion réseau** (modération et notifications se font sur le serveur) : limite assumée, à expliquer à la soutenance.
- L'auteur **suit automatiquement** son post.
- **Modification** réservée à l'auteur. Après **30 minutes**, une modification affiche le marqueur « edited ». Un fil verrouillé ne peut plus être modifié ni recevoir de réponse.
- **Réponses** : une par minute et par membre (429 sinon).
- **Commentaires** : un seul niveau (un commentaire répond à une réponse, pas à un commentaire), 500 caractères maximum.
- Répondre met à jour `replyCount` et `lastActivityAt`, et abonne le membre au fil, sauf s'il l'a mis en sourdine.
- **Votes** : un par membre et par élément, annulable (bascule atomique côté base).
- **Signalement** : un signalement ouvert par membre et par élément ; il envoie l'élément dans la file de modération.
- **Suppression** par l'auteur ou un administrateur ; supprimer une réponse supprime ses commentaires et corrige le compteur.

Modération (administrateurs)

- File des posts et réponses signalés, avec les motifs et leurs auteurs.
- Actions : masquer / réafficher, verrouiller / déverrouiller le fil, supprimer, ignorer les signalements.
- Un post masqué reste en base avec `status = hidden` (exigence du cahier des charges). Toute action clôt les signalements.

Logique de la fonctionnalité avancée — Recherche plein texte et notifications Web Push

Recherche plein texte : index `$text` pondéré, tri par score de pertinence, combinable avec les filtres catégorie et tag.

Notifications Web Push (clés VAPID) : à chaque nouvelle réponse, une notification est envoyée à tous les abonnés du fil, sauf à l'auteur de la réponse, à ceux qui ont désactivé les notifications du forum et à ceux qui ont mis ce fil en sourdine.

Un utilisateur peut avoir **plusieurs appareils**. Les abonnements expirés (réponses 404 ou 410 du service push) sont supprimés automatiquement.

« Unfollow » = **sourdine du fil** : on ne reçoit plus rien et on n'est plus abonné automatiquement en répondant. Un réglage global existe aussi dans le profil.

La **permission est demandée en contexte** (après avoir publié, répondu ou suivi un fil), **jamais au premier chargement**.

Le service worker affiche la notification ; un clic ouvre directement la bonne réponse.

Hors ligne : lecture des **50 derniers posts ouverts** (cache du service worker) ; écriture impossible.